

iOS Application Development – Fundamentals

Day 04 - Session 1

24th September 2026,
Thursday

Agenda

01

iOS Ecosystem & Architecture

02

Swift Fundamentals

03

View/UI Architecture

04

Entitlements & Permissions

05

Secure Storage

06

Application Transport Security

iOS Ecosystem & Architecture

iOS Ecosystem

01

iOS Definition

Apple's proprietary mobile operating system built specifically to power the **iPhone**.

03

Key Benefits of iOS

- Deep hardware-software optimization
- Strict privacy and security
- Seamless ecosystem synergy



02

History & Creator

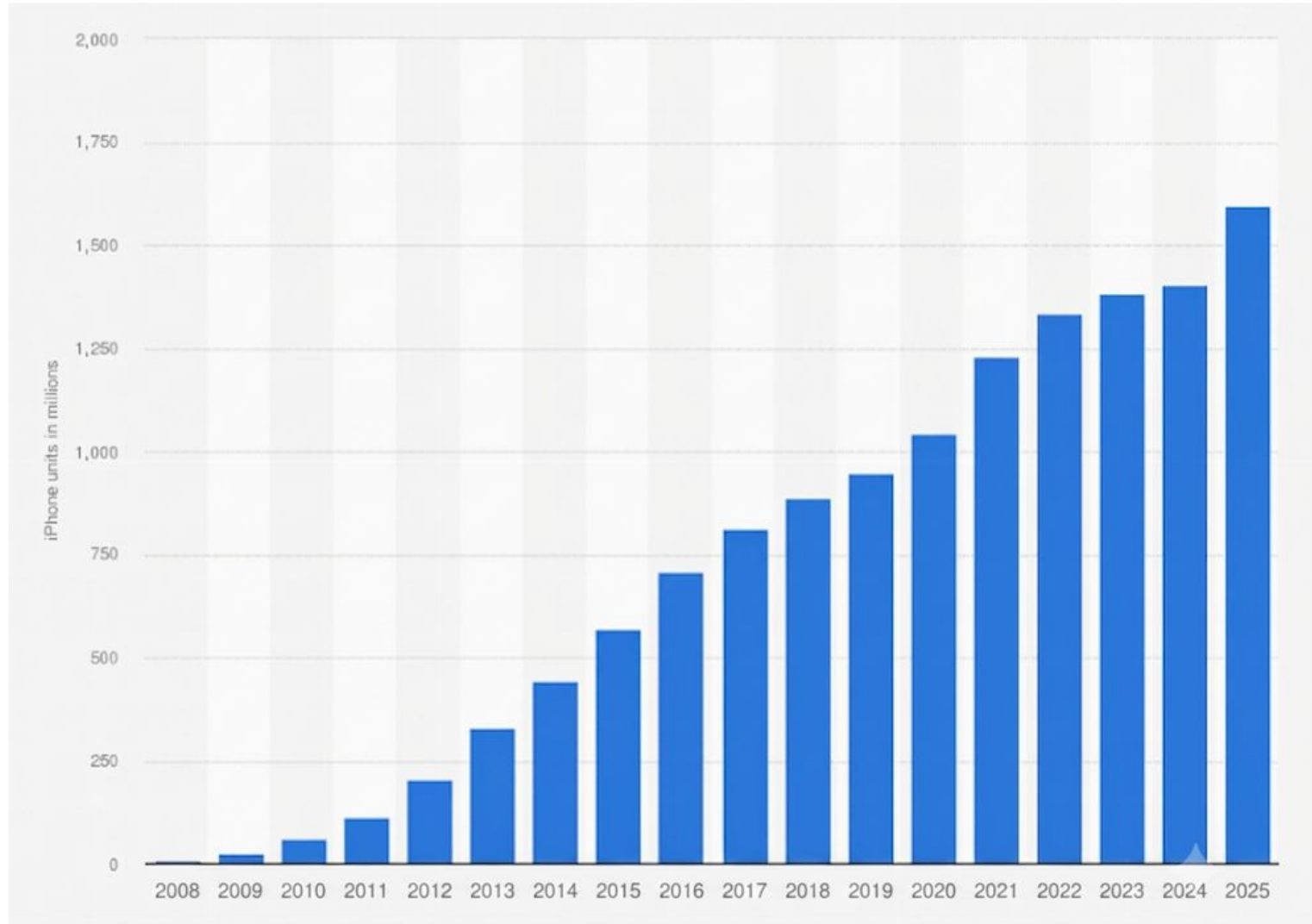
Developed by **Apple Inc.** under Steve Jobs and released in **June 2007** as iPhone OS.

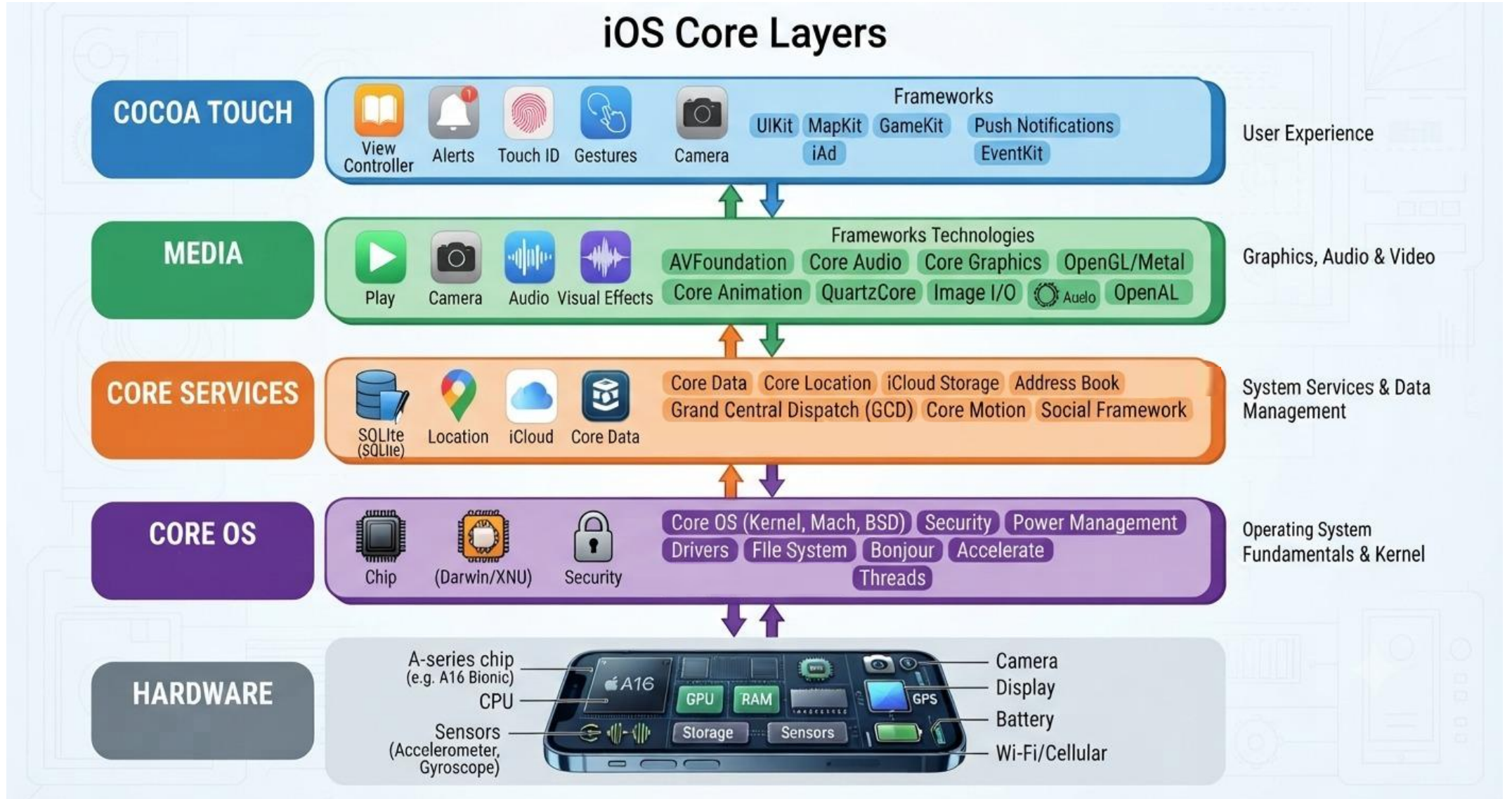
04

Market Statistics

~1.56 billion active users globally, serving as the world's **second-largest** mobile OS after Android.

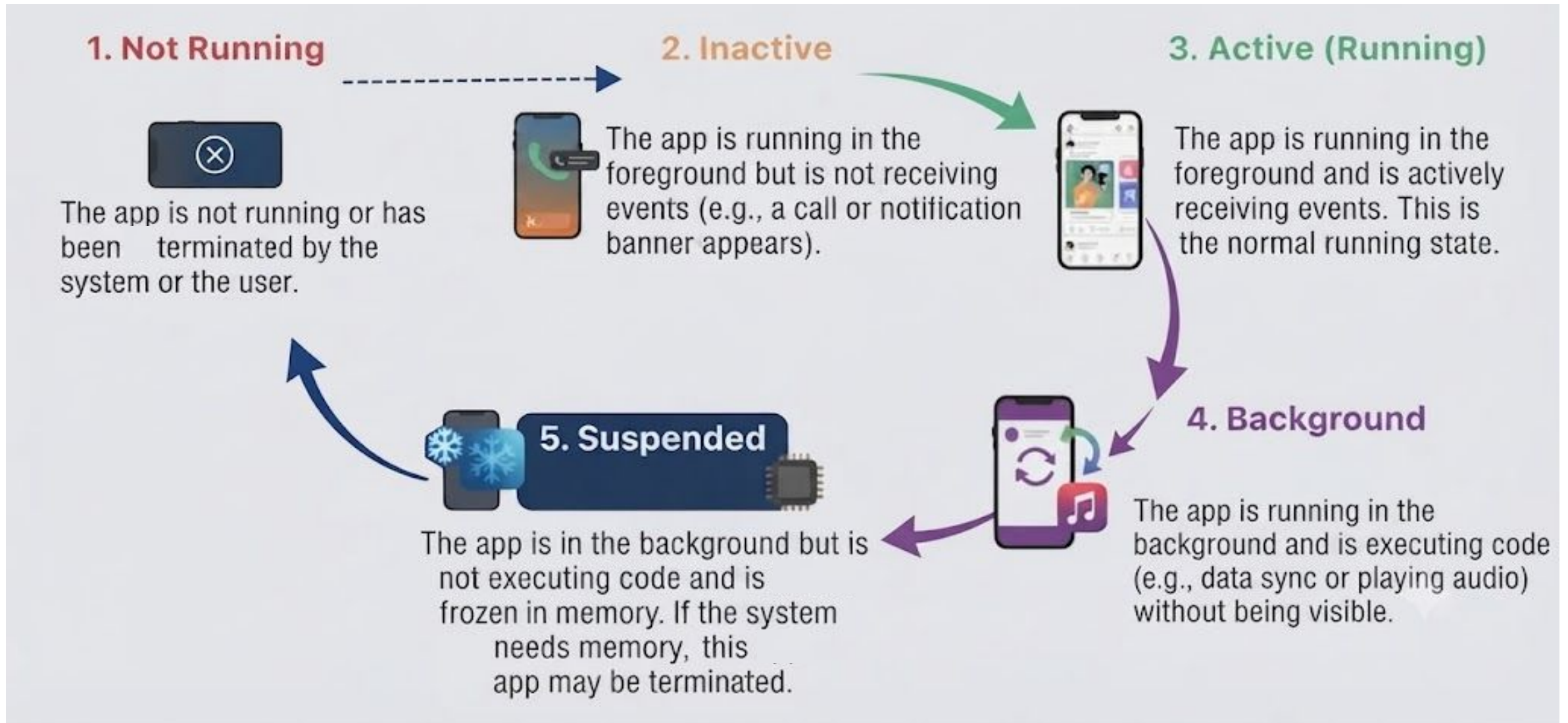
Evolution of Global iPhone Adoption





iOS Security Architecture





iOS Application Sandboxing

Core Isolation Features

- **Strict File Isolation**
- **No Direct Hardware Access**
- **Inter-App Restrictions**



Core Objectives

- **Containment:** Limits security breaches to the affected app
- **Privacy:** Blocks unauthorized access to personal user data
- **Stability:** Protects core OS files from unauthorized modifications

Controlled External Access

- **Runtime Permissions:** Interactive user prompts at runtime
- **Security-Scoped URLs:** Temporary access to selected files/folders

Swift Fundamentals

Introduction To Swift

Overview

A modern, high-performance, and safe programming language created by Apple.

Timeline: Developed starting in 2010, publicly released in 2014, and made open-source in 2015.

Platforms

Primarily used for iOS, macOS, watchOS, and tvOS development, as well as server-side backend services.

iOS

macOS

watchOS

tvOS

Server-Side Backend

Advantages

Safety

First-class optionals eliminate null-pointer crashes.

Modern Syntax

Clean, expressive, and easy to maintain with modern, developer-friendly syntax.

Efficient Memory Management

Efficient, deterministic memory management without a garbage collector.

First-Class Optionals

Safety Over Nulls

Swift enforces safe, distinct types for missing values at compile-time instead of risking runtime null exceptions.

Execution Guard: Forces developers to handle **nil** cases before execution.

Type System

Optional types (**Int?**) are distinct from non-optional types (**Int**).

Handles Absence of Value

An **Optional** (?) either contains a valid value or is completely empty (**nil**).

```
Options.swift

// Non-optional cannot be nil
var count: Int = 10

// Optional type declaration
var score: Int? = 100

// Can safely store nil (no value)
score = nil

// Compiler forces safe unwrapping
if let validScore = score {
    print(validScore)
} else {
    print("No value present")
}
```

Key Concepts In Swift

Value Types

Heavy Preference for Value Types: Employs **struct** copy-on-assignment semantics to eliminate unintended side effects and ensure thread safety.

POP Architecture

Protocol-Oriented Programming (POP): Builds flexible, modular code through lightweight protocol composition and extensions instead of rigid class inheritance.

Memory Management

Automatic Reference Counting (ARC): Manages memory deterministically without a garbage collector by instantly freeing references and using explicit weak capture lists.

View / UI Design Architecture

View / UI Architecture

View/UI architecture is the set of patterns and frameworks that answer how an app manages user interface display and event processing.

Every iOS app needs to solve two core problems:

- **Display Logic**
- **User Event Handling**



UIKit

The Original Framework

The traditional imperative UI framework for iOS, providing event handling and UI components.



SwiftUI

The Modern Successor

The modern declarative UI framework designed for building user interfaces across all Apple platforms seamlessly.

UIKit — The Imperative Model

Imperative Approach

Writing step-by-step instructions telling the UI exactly what to do and when.

```
override func viewDidLoad() {  
    nameLabel.text = user.name // manual push  
}  
  
@objc func editTapped() {  
    user.name = "New Name"  
    nameLabel.text = user.name  
    // remember to update view again  
}
```

UIView

01

Draws pixels and handles user interaction (buttons, labels, images).

UIViewController

02

Manages a screen's complete view hierarchy and lifecycle events.

Layouts

03

Built visually using Interface Builder or programmatically in code.

SwiftUI – The Declarative Model

Declarative Approach

- **Declarative UI:** Focus on “**what**” to display.
- **Automatic Re-rendering:** State changes drive UI updates seamlessly.

```
struct ProfileView: View {  
    @State private var name = "Jane"  
    var body: some View {  
        VStack {  
            Text("Hello, \(name)")  
            Button("Edit") { name = "New Name" } // state mutation  
        }  
    }  
}
```

@State

01

A view's own private, local state

@Binding

02

A two-way value link to state owned by a parent view.

@Observable / @ObservedObject

03

Shared state living in an external reference object (ViewModel).

SwiftUI – Layout Stacks (HStack, VStack, ZStack)

SwiftUI layout stacks position child views along different axes.

HStack — Positions child views in a horizontal row side-by-side.

VStack — Positions child views in a vertical column from top to bottom.

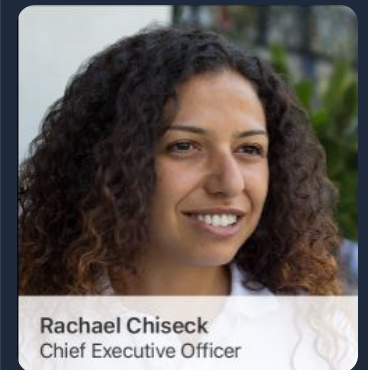
ZStack — Overlays child views on top of one another along the Z-axis.



SwiftUI Code Implementation

```
struct ProfileView: View {  
    var body: some View {  
        ZStack(alignment: .bottom) {  
            Image("ProfilePicture")  
                .resizable()  
                .aspectRatio(contentMode:  
                    .fit)  
            HStack {  
                VStack(align: .leading) {  
                    Text("Rachael...")  
                        .font(.headline)  
                    Text("CEO")  
                }  
                Spacer()  
            }  
        }  
        .padding()  
        .background(Color...)  
    }  
}
```

// Output



SwiftUI – Working with Buttons

In **SwiftUI**, interactive buttons are declared using the **Button** view.

- **Declarative Action:** Handles user taps seamlessly.
- **Customizable Label:** Supports text, icons, or complex layouts.

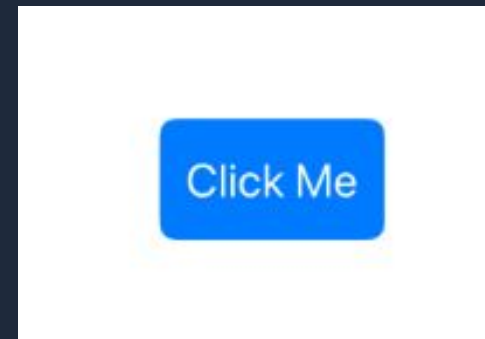


SwiftUI Code Implementation

```
import SwiftUI

struct SimpleButtonView: View {
    var body: some View {
        Button("Click Me") {
            // This code runs when the button is clicked
            print("Button was clicked!")
        }
    }
}
```

//OUTPUT:



SwiftUI – Displaying Basic Text

- Simply declare what string value to present.
- Chain built-in methods directly to customize size, weight, and color.
- Combine with stacks (e.g., **VStack**) to arrange multiple text views.

Common Styling Modifiers

- **.font(.title)** – Sets dynamic title size.
- **.fontWeight(.bold)** – Makes text bold.
- **.foregroundColor(.blue)** – Colors text blue.
- **.padding()** – Adds surrounding space.



SwiftUI Code Implementation

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        VStack(spacing: 15) {
            // The simplest way to display text
            Text("Hello, World!")

            // Text with basic styling modifiers
            Text("Styled SwiftUI Text")
                .font(.title) // Title preset
                .fontWeight(.bold) // Bold text
                .foregroundColor(.blue) // Blue color
        }
        .padding()
    }
}

#Preview {
    ContentView()
}
```

// OUTPUT:

Hello, World!

Styled SwiftUI Text

SwiftUI – Displaying & Scaling Images

1. Basic Image Rendering

By default, SwiftUI renders an image at its original pixel dimensions.

- **Unconstrained Size**
- **No Auto-Fit**

2. Resizable & Constrained Image (Recommended)

- **.resizable()** – Scales the image dynamically.
- **.scaledToFit()** – Preserves aspect ratio.
- **.frame(w, h)** – Caps max width & height dimensions.
- **.clipShape(...)** – Clips view into rounded shapes or custom geometry.



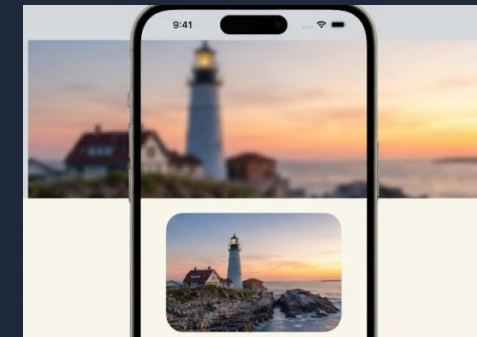
SwiftUI Code Implementation

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        // 1. Basic image (renders unscaled)
        Image("myImage")

        // 2. Resizable & constrained image
        Image("myImage")
            .resizable() // Allows scaling
            .scaledToFit() // Maintains ratio
            .frame(width: 300, height: 200)
            .clipShape(RoundedRectangle(cornerRadius: 15))
    }
}
```

//OUTPUT:



Entitlements & Permissions

Entitlements & Permissions

Entitlements are built-in permissions granted by Apple, while runtime permissions are pop-up prompts asking users for access while using the app.

Entitlements **Build-time Capabilities**

Developer enables the capability in App Configuration

 Push Notifications

 iCloud

 App Groups

 HealthKit

☐ Sign in with Apple

Runtime Permissions **User Authorization**

User grants explicit permission at app runtime

 Camera

 Microphone

 Location

 Photos

 Contacts

Info.plist: Purpose Strings & Permissions

User Transparency

Info.plist Purpose Strings

Mandatory explanation (**NSUsageDescription**) shown to user.

! Observation

App Crash Risk

Requesting permissions without purpose string crashes the app immediately.

Runtime permissions requiring Purpose Strings:

- Camera (e.g., scanning receipts)
- Location services
- Contacts access
- And other sensitive hardware/data permissions, etc.

```
Info.plist & Access Request Implementation

<!-- Info.plist -->
<key>NSCameraUsageDescription</key>
<string>Used to scan receipts</string>

<!-- App.entitlements -->
<key>com.apple.developer.icloud-
  container-identifiers</key>
<array><string>iCloud.com.app.id</string></array>

// Swift - requesting camera access
AVCaptureDevice.requestAccess(for: .video) {
    granted in
    guard granted else { return }
    startCameraSession()
}
```

Secure Storage

SECURE STORAGE: Choosing the Right Storage Option



1. UserDefaults

LOW SECURITY (UNENCRYPTED)

Saves data as a plain-text plist file.

Suitable For:

- **App settings:**
 - font size
 - Volume levels
- **Toggle flags:**
 - Dark Mode
 - notification
- **User preferences:**
 - Grid or List view choice
 - sort by date or name



2. Keychain

HIGH SECURITY (HARDWARE)

Uses hardware-backed encryption for extreme security.

Suitable For:

- Credentials & passwords
- Access tokens
- Security keys



Biometric Protection:

Can be locked behind biometric gate (Face ID / Touch ID).



3. File Protection

OS CLASS-KEY ENCRYPTION

Encrypts files at rest using OS class-specific keys.

Suitable For:

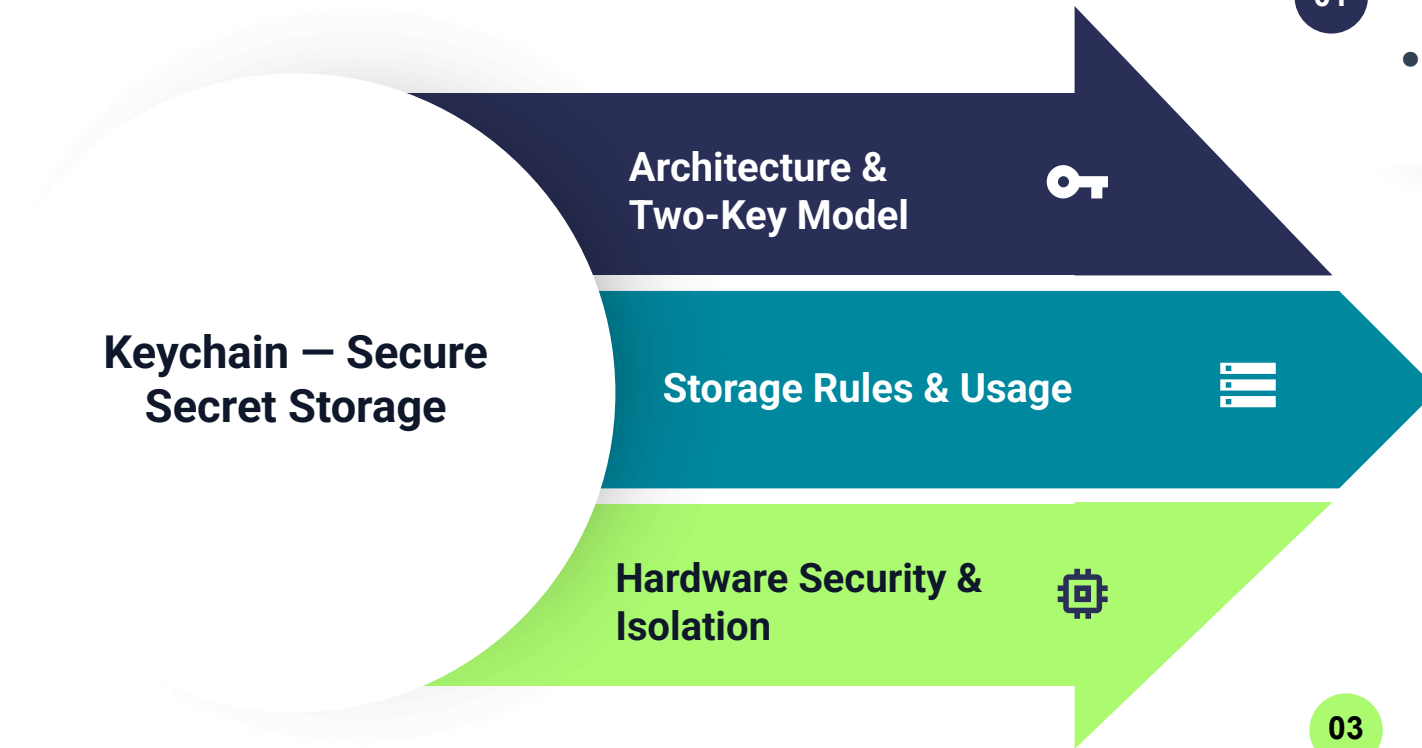
- Documents
- Media files
- Cache files



Key Protection Modes:

- **complete:**
- **completeUnlessOpen**
- **completeUntilFirstUser(Default)**

Keychain - Secure Storage



01

- Encrypted system database for sensitive iOS data.
- **Two-Key Model:** Uses a device UID key and unique item metadata keys.
- **Practical Security:** Backups extracted without the device UID key remain useless ciphertext.

02

Rule: Never store sensitive information in plain text files or UserDefaults.

- **Passwords & PIN codes**
- **Auth Tokens**
- **Payment & Biometrics**

03

- Tied directly to the device's secure processor keys
- Restricts data access exclusively to the app
- **Device Lock Protection**

App Transport Security (ATS)

APP TRANSPORT SECURITY (ATS)

A security feature that forces iOS apps to connect only through secure HTTPS and TLS connections

01



HTTPS Protocol

It securely transmits data between a user's browser and a website.

02



Modern TLS (Transport Layer Security)

secures and encrypts all internet communications to guarantee data privacy and integrity.

03



Forward Secrecy

Uses unique session keys, so past communications stay secure even if the server's main key is stolen.

04



Trusted Certificates

Verified digital certificates from recognized authorities that devices use to confirm server identity.

ATS Configuration Risks

HIGH RISK – Core Encryption Disabled

These configurations completely disable the core encryption and security boundaries, leaving traffic entirely unencrypted and open to interception.

NSAllowsArbitraryLoads: true

Allows unencrypted HTTP traffic and removes all TLS security rules.

Disables ATS globally

HIGH RISK

NSAllowsArbitraryLoadsInWebContent: true

Enables unencrypted HTTP traffic, exposing in-app views to tampering and eavesdropping.

HIGH RISK

NSExceptionAllowsInsecureHTTPLoads: true

Permits unencrypted HTTP traffic to specific domains.

HIGH RISK

MEDIUM RISK – Cryptographic Compromise

These configurations do not necessarily open up all traffic to plain HTTP, but instead compromise cryptographic strength or validation layers.

NSExceptionMinimumTLSVersion

Permits weak TLS versions for specific domains, risking protocol downgrade attacks.

MEDIUM RISK

NSRequiresCertificateTransparency: false

Bypasses Certificate Transparency (CT) log checks, hiding fraudulent certificates and making rogue issues harder to detect.

MEDIUM RISK

Certificate Transparency

Disabling CT checks leaves the app vulnerable to targeted MitM attacks using fraudulent certificates issued by rogue authorities.

MEDIUM RISK

ATS CONFIGURATION (Info.plist)

ATS configurations are managed in the Info.plist file's NSAppTransportSecurity dictionary; overriding defaults poses security risks.

```
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSAllowsArbitraryLoads</key>
  <false/>
  <key>NSExceptionDomains</key>
  <dict>
    <key>yourdomain.com</key>
    <dict>
      <key>NSExceptionMinimumTLSVersion</key>
      <string>TLSv1.2</string>
      <key>NSIncludesSubdomains</key>
      <true/>
    </dict>
  </dict>
</dict>
```

ATS MITIGATION

Key practices to maintain end-to-end network encryption and security

01

Default ATS Configurations

Enforce HTTPS and TLS 1.2+ end-to-end by keeping default ATS configurations.

02

Domain Exceptions

Narrowing domain exceptions to reduce vulnerable attack surfaces.

03

Periodic Audits

Conduct periodic audits to ensure continuous compliance and identify potential risks.

THANK YOU !